CAHIER DES CHARGES TECHNIQUE

Projet : Cartes grises

Langages : MySQL – Java

Méthodologie : Agile Kanban (Trello)

Imad Chattou

BTS SIO – 2025/2026

Sommaire

1. Contexte du projet
    1.1. Présentation du projet
    1.2. Date de rendu du projet
2. Besoins fonctionnels
3. Ressources nécessaires à la réalisation du projet
    3.1. Ressources matérielles
    3.2. Ressources logicielles
4. Gestion du projet
5. Gestion du projet
6. Conception du projet
    6.1. Le front-end
        6.1.1. Wireframes
        6.1.2. Maquettes
        6.1.3. Arborescences
    6.2. Le back-end
        6.2.1. Diagramme de cas d'utilisation
        6.2.2. Diagramme de classe
        6.2.3. Diagramme d'activités
        6.2.4. Modèles Conceptuel de Données (MCD)
        6.2.5. Modèle Logique de Données (MLD)
        6.2.6.Modèle Physique de Données (MPD)
7. Technologies utilisées
    7.1. Langages de développement Web
    7.2. Base de données
    8. Sécurité
    8.1. Main : Initialisation et contrôle d'accès de l'application
    8.2. MVC (Modèle-Vue-Contrôleur)
    8.3. Protection contre les injections SQL
    8.4. CRUD (Create, Read, Update, Delete)

1. Contexte du projet

1.1. Présentation du projet

La Direction des systèmes d'information de la préfecture a été sollicitée par le service des cartes grises afin de faire évoluer leur application métier actuelle, permettant la gestion des dossiers de cartes grises.

Votre équipe et vous-même avez pour mission d'analyser l'application existante, de produire les documents de conception nécessaires, puis de proposer et mettre en œuvre une solution répondant aux besoins d'évolution exprimés par le service.

1.2. Date de rendu du projet

Le projet doit être rendu au plus tard le 15/01/2025.

2. Besoins fonctionnels

Le site web devra avoir une partie accessible au public et une partie privée permettant de gérer les données.

Les données seront stockées dans une base de données relationnelle pour faciliter la gestion et la mise à jour des informations. Ces données peuvent être gérées directement via le site web à travers un espace administrateur.

3. Ressources nécessaires à la réalisation du projet

3.1. Ressources matérielles

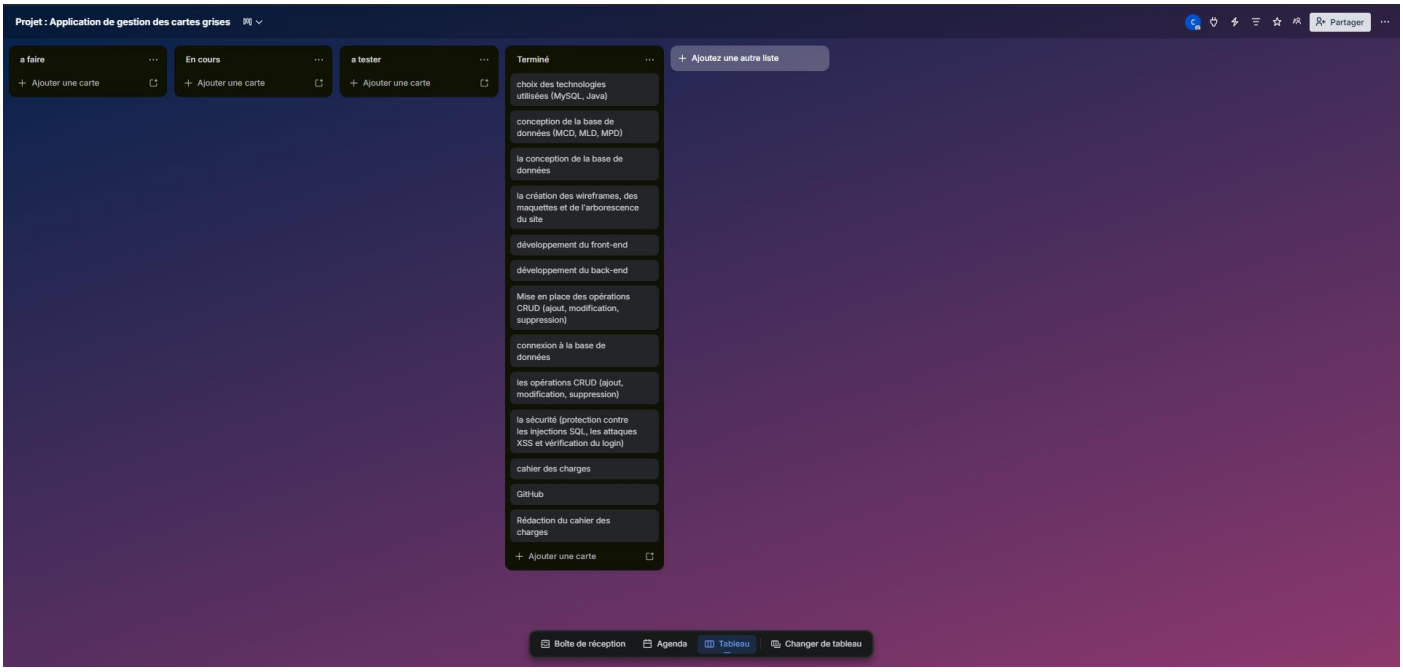
- PC
- Connexion internet
- Serveur local
- Périphérique de travail

3.2. Ressources logicielles

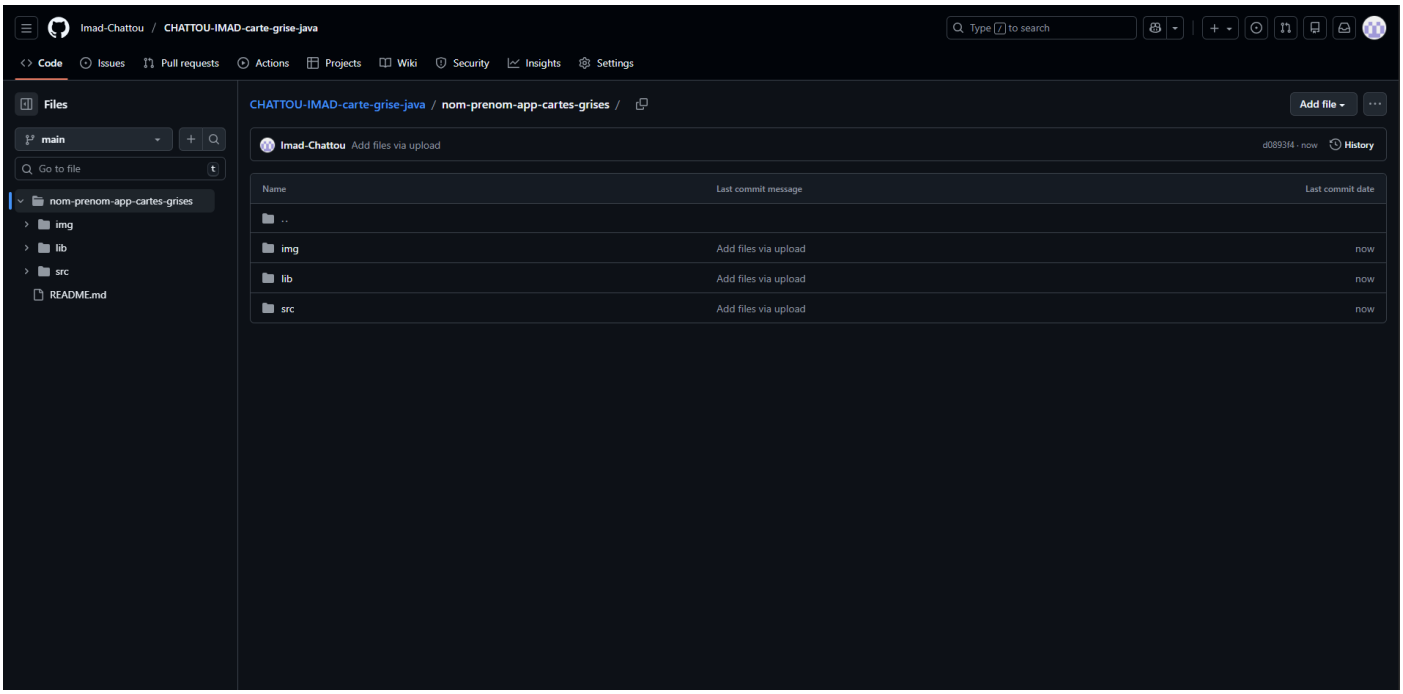
- Visual studio code
- Mysql
- Mamp
- Mocodo
- Java

4. Gestion du projet

Pour réaliser le projet, nous utiliserons la méthode Agile Kanban. Nous utiliserons également l'outil de gestion de projet en ligne Trello.



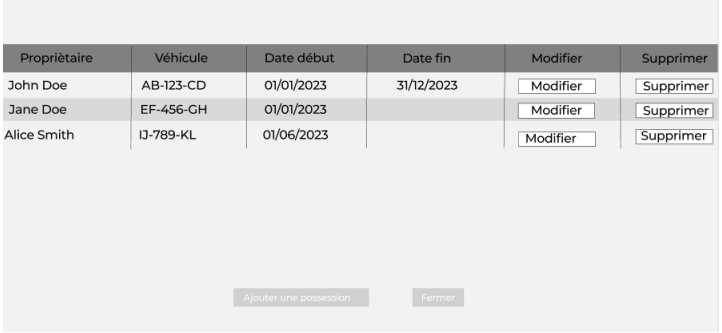
5. Partage collaboratif sur GitHub

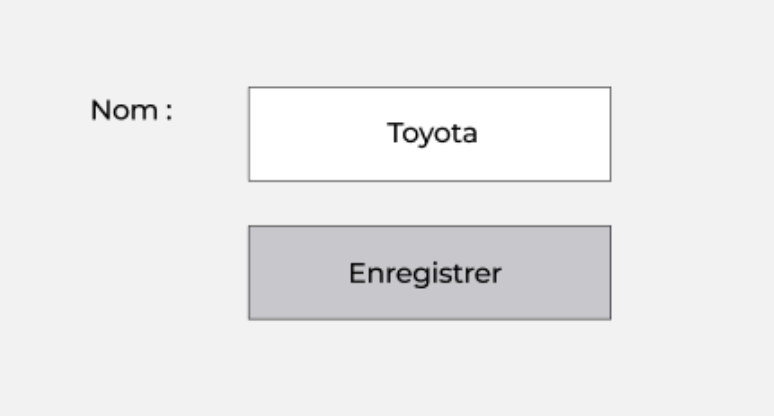


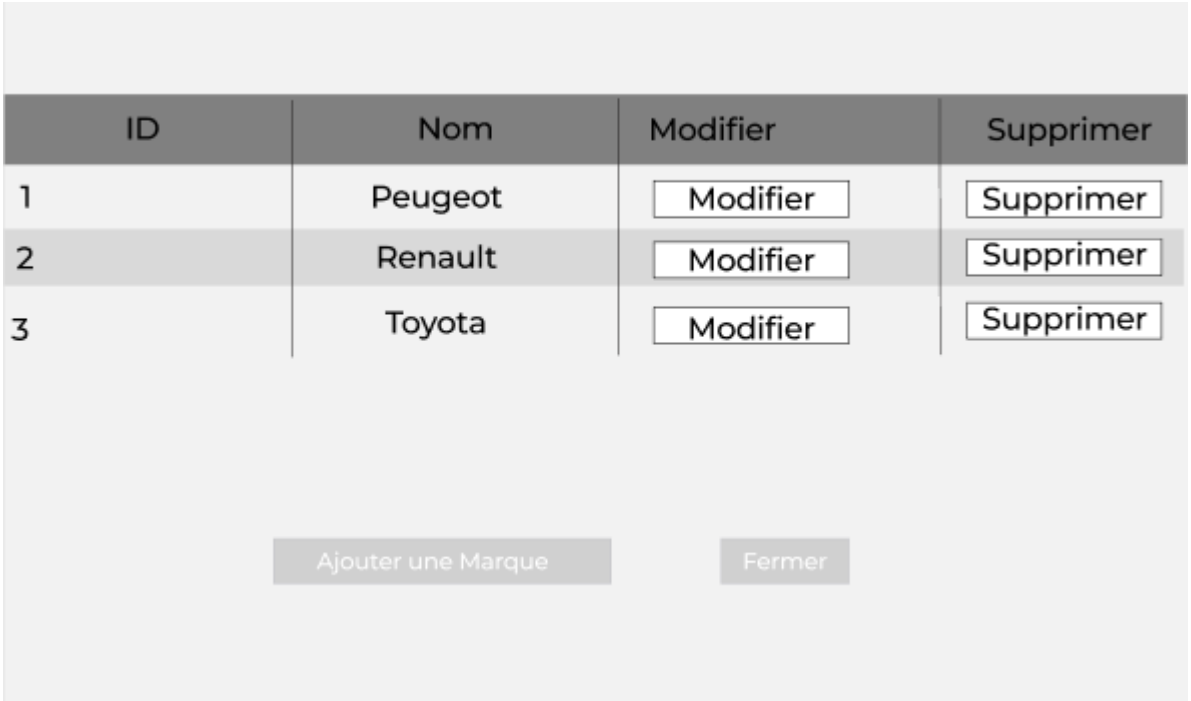
6. Conception du projet

6.1. Le front-end

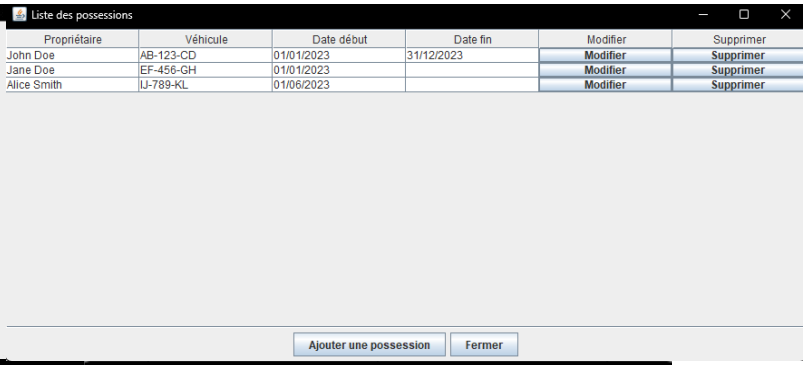
6.1.1. Wireframes

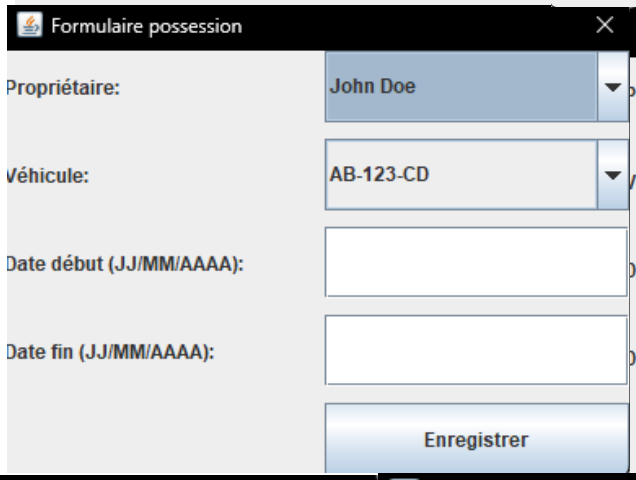


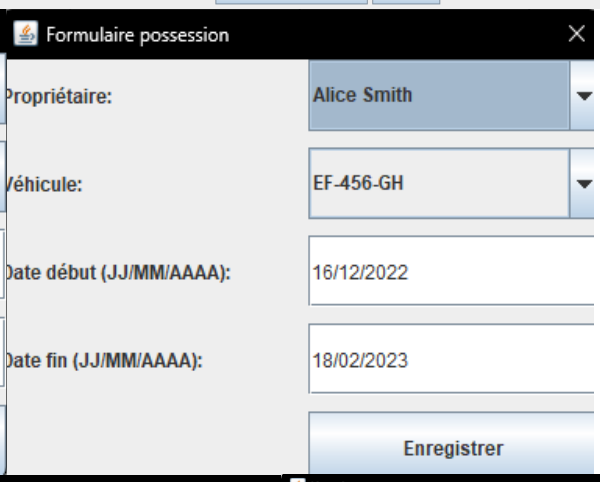


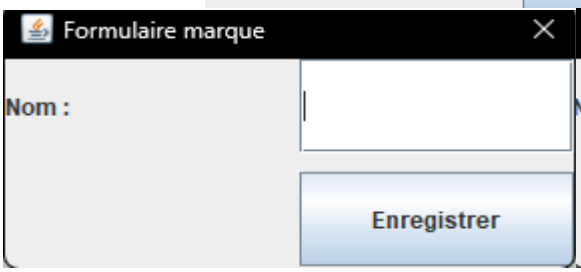


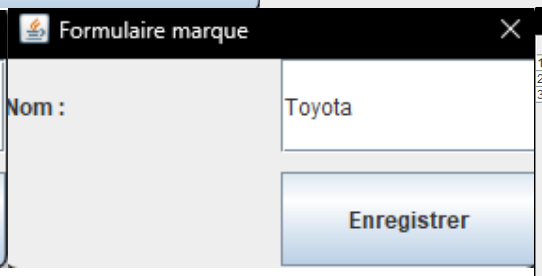
6.1.2. Maquettes



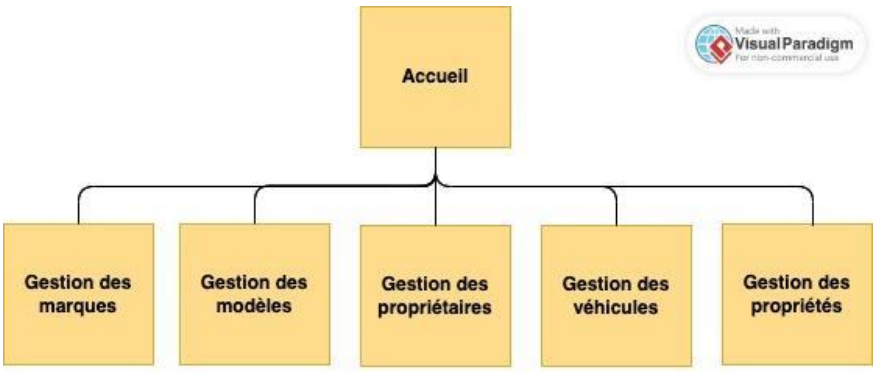






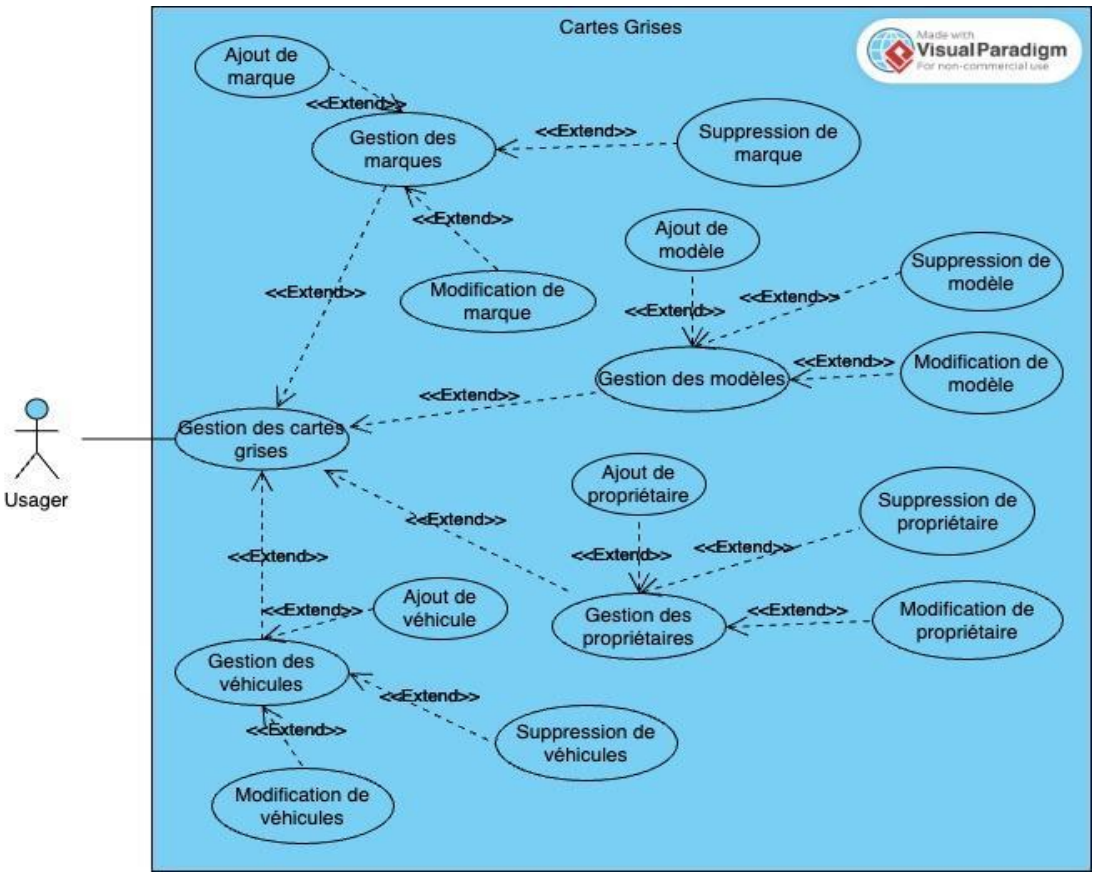


6.1.3. Arborescences

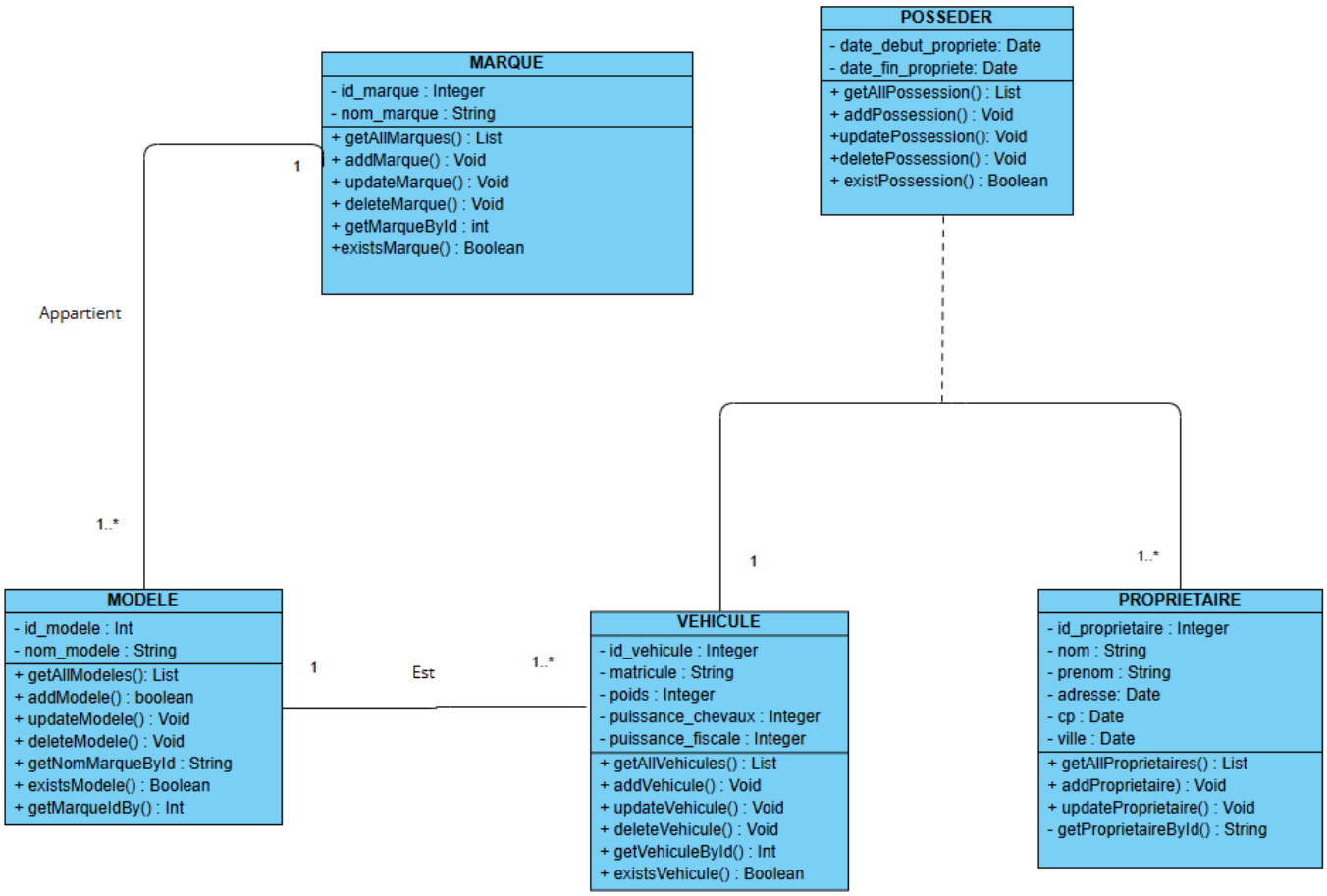


6.2. Le back-end

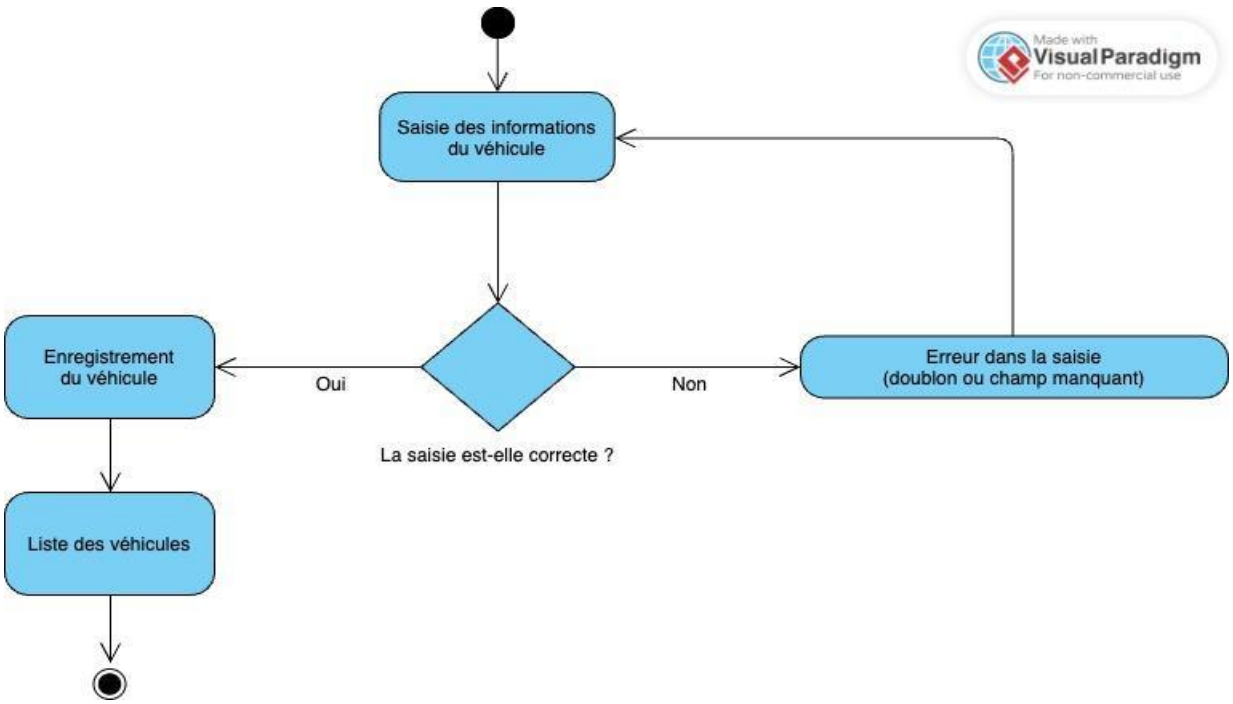
6.2.1. Diagramme de cas d'utilisation



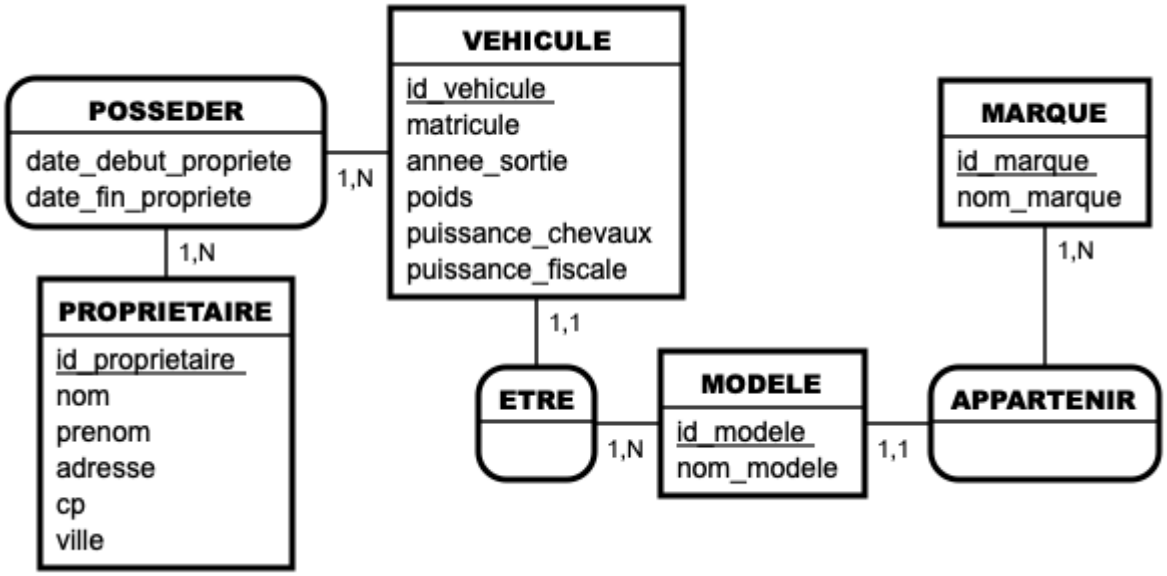
6.2.2. Diagramme de classe



6.2.3. Diagramme d'activités



6.2.4. Modèles Conceptuel de Données (MCD)



6.2.5. Modèle Logique de Données (MLD)

- **MARQUE (id_marque, nom_marque)**
  Clé primaire :id_marque

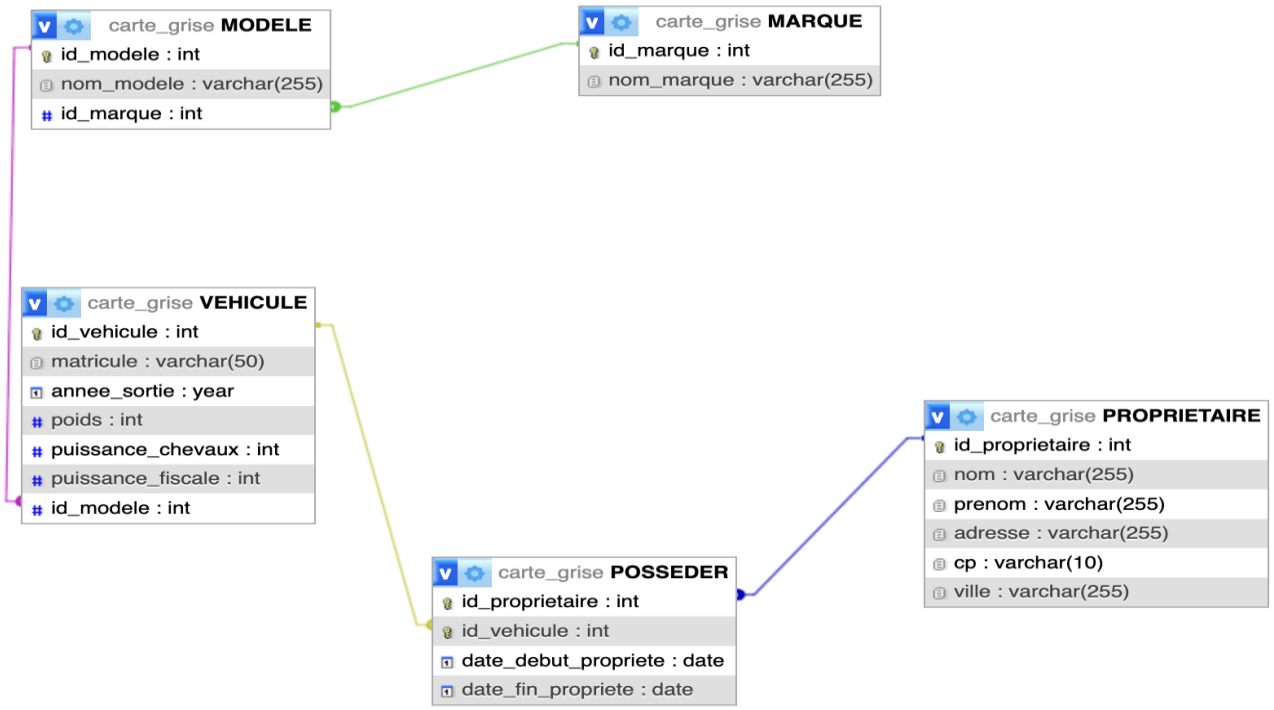
- **MODELE (id_modele, nom_modele, id_marque)**
  Clé primaire :id_modele
  Clé étrangère :id_marque en référence à id_marque de MARQUE

- **VEHICULE (id_vehicule, matricule, annee_sortie, poids, puissance_chevaux, puissance_fiscale, id_modele)**
  Clé primaire : id_vehicule
  Clé étrangère :id_modele en référence à id_modele de MODELE

- **PROPRIETAIRE (id_proprietaire, nom, prenom, adresse, cp, ville)**
  Clé primaire :id_proprietaire

- **POSSEDER (id_proprietaire, id_vehicule, date_debut_propriete, date_fin_propriete)**
  Clé primaire :id_proprietaire, id_vehicule
  Clés étrangères :id_proprietaire en référence à id_proprietaire de PROPRIETAIRE id_vehicule en référence à id_vehicule de VEHICULE

6.2.6. Modèle Physique de Données (MPD)



7. Technologies utilisées

7.1. Langages de développement Web

7.2. Base de données

8. Sécurité

8.1. Main : Initialisation et contrôle d'accès de l'application

Définition de la solution :

La classe Main constitue le point d'entrée de l'application. Sa mission principale est de lancer l'application Swing dans le thread approprié, de créer et initialiser tous les contrôleurs (Marque, Modèle, Véhicule, Propriétaire, Posséder) et de charger la vue principale (MainView) qui sert de menu central.
Elle peut également être adaptée pour intégrer un système de login ou de vérification des droits administrateurs avant l'accès aux fonctionnalités sensibles, assurant ainsi la protection des pages critiques.
En résumé, Main assure la coordination initiale entre la logique métier (contrôleurs) et l'interface utilisateur (vue principale), garantissant que l'application démarre correctement et de manière sécurisée.

Extraits de code

```
        // Pour que l'interface Swing se lance dans le bon thread
        SwingUtilities.invokeLater(() -> {
            // Création des contrôleurs
            MarqueController marqueController = new MarqueController();
            ModeleController modeleController = new ModeleController();
            VehiculeController vehiculeController = new
VehiculeController();
            ProprietaireController proprietaireController = new
ProprietaireController();
            PossederController possederController = new
PossederController();

            // Création de la vue principale (menu)
            MainView mainView = new MainView(
                    marqueController,
                    modeleController,
                    vehiculeController,
                    proprietaireController,
                    possederController
            );
}
```

Pourquoi cet extrait est pertinent :

- Il montre que Main initialise tous les contrôleurs, c'est-à-dire la logique métier de ton application.
- Il montre que Main lance la vue principale (MainView), qui est l'interface graphique de l'application.
- Il utilise SwingUtilities.invokeLater, ce qui indique que Swing est utilisé pour gérer l'interface graphique dans le bon thread.

8.2. MVC (Modèle-Vue-Contrôleur)

Définition de la solution :

Le MVC est un patron de conception qui sépare une application en trois parties :

Extraits de code

Modèle (Model) :

- Rôle : Gère les données et la logique métier.
- Dans ton projet : Proprietaire
- Fonctions : CRUD (Create, Read, Update, Delete), accès à la base de données via JDBC, validation des données côté serveur.

```
// Ajout sécurisé d'un propriétaire (protection contre les injections SQL)
public static boolean addProprietaire(String nom, String prenom, String
adresse, String cp, String ville) {
    String sql = "INSERT INTO PROPRIETAIRE (nom, prenom, adresse, cp, ville)
VALUES (?, ?, ?, ?, ?)";
    try (Connection conn = DBConnection.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, nom);
        ps.setString(2, prenom);
        ps.setString(3, adresse);
        ps.setString(4, cp);
        ps.setString(5, ville);
        return ps.executeUpdate() > 0;
    } catch (SQLException e) { return false; }
}
```

Vue (View) :

- Rôle : Interface graphique et interaction avec l'utilisateur.
- Dans ton projet : ProprietaireView, MainView
- Fonctions : Affiche les tableaux, formulaires et boutons, récupère les actions de l'utilisateur et les transmet au contrôleur.

```
JButton addButton = new JButton("Ajouter un propriétaire");
addButton.addActionListener(e -> showProprietaireForm(null));

JButton closeButton = new JButton("Fermer");
closeButton.addActionListener(e -> dispose()); // ferme la fenêtre
CONFIRMATION D'ACTION (Protection indirecte) :
// Dans ButtonEditor class - ligne ~175
int confirm = JOptionPane.showConfirmDialog(ProprietaireView.this,
    "Supprimer ce propriétaire ?",  // ← Message fixe, sécurisé
    "Confirmer",
    JOptionPane.YES_NO_OPTION);
```

Contrôleur (Controller) :

Rôle : Intermédiaire entre la Vue et le Modèle.
Dans ton projet : ProprietaireController
Fonctions : Récupère les données du modèle, les transmet à la vue, applique la logique métier, et déclenche les modifications.

```
// Récupérer tous les propriétaires pour les afficher dans la JTable
public List<Proprietaire> fetchAllProprietaires() {
    return Proprietaire.getAllProprietaires();
}

// Ajouter un propriétaire depuis la vue
public boolean createProprietaire(String nom, String prenom, String adresse,
String cp, String ville) {
    return Proprietaire.addProprietaire(nom, prenom, adresse, cp, ville);
}
```

Résumé graphique du MVC dans ton projet :

[Vue] ProprietaireView / MainView
↑ ↓
[Contrôleur] ProprietaireController
↑ ↓
[Modèle] Proprietaire (CRUD + DB)

- La Vue : interface Swing (tableaux, boutons, formulaires).
- Le Contrôleur : gère les actions et interactions.
- Le Modèle : gère les données et la base MySQL.

8.3. Protection contre les injections SQL

Définition de la solution :

Injection SQL : Attaque de sécurité qui permet à un utilisateur malveillant d'interférer avec les requêtes SQL qu'une application exécute sur sa base de données.
Comment ça marche ?
L'attaquant insère du code SQL malveillant dans des champs de saisie (formulaires, URL, etc.) qui est ensuite exécuté par la base de données lorsqu'il est concaténé sans protection dans une requête SQL.
Conséquences possibles :

- Accès non autorisé aux données
- Modification/suppression de données
- Élévation de privilèges
- Exécution de commandes système

Extraits de code

Extrait : CRUD sécurisé avec protection SQL (Proprietaire / modèle)

```
// Ligne public static boolean addProprietaire(String nom, String prenom,
String adresse, String cp, String ville) {
    String sql = "INSERT INTO PROPRIETAIRE (nom, prenom, adresse, cp, ville)
VALUES (?, ?, ?, ?, ?)";

    try (Connection conn = DBConnection.getConnection();
         PreparedStatement ps = conn.prepareStatement(sql)) {

        ps.setString(1, nom);
        ps.setString(2, prenom);
        ps.setString(3, adresse);
        ps.setString(4, cp);
        ps.setString(5, ville);

        return ps.executeUpdate() > 0;

    } catch (SQLException e) {
        System.err.println("Erreur addProprietaire : " + e.getMessage());
        return false;
    }
}
```

Extrait : Interface graphique et interaction avec le contrôleur (ProprietaireView) :

```
JButton addButton = new JButton("Ajouter un propriétaire");
addButton.addActionListener(e -> showProprietaireForm(null));

JButton closeButton = new JButton("Fermer");
closeButton.addActionListener(e -> dispose()); // ferme uniquement cette
fenêtre
```

TECHNIQUES UTILISÉES DANS VOTRE CODE :

- Illustre l'utilisation de PreparedStatement pour éviter les injections SQL.
- Montre comment le modèle gère l'ajout de données dans la base de données de façon sécurisée.
- Montre comment la vue Swing interagit avec le contrôleur pour ajouter un nouveau propriétaire.
- Illustre la séparation MVC : la vue ne fait que déclencher les actions, le contrôleur gère la logique métier.

8.4. CRUD (Create, Read, Update, Delete)

Le CRUD désigne les quatre opérations de base pour gérer les données dans une application. Dans ton projet, il concerne la gestion des propriétaires de véhicules via la base MySQL.

Extraits de code

Create (Créer) :

- Rôle : Ajouter de nouvelles données dans la base.
- Dans ton projet : Proprietaire.addProprietaire() / ProprietaireController.createProprietaire()

Extrait de code :

```
// Ajouter un nouveau propriétaire
public boolean createProprietaire(String nom, String prenom, String adresse,
String cp, String ville) {
    return Proprietaire.addProprietaire(nom, prenom, adresse, cp, ville);
}
```

Read (Lire) :

- Rôle : Récupérer des données existantes pour les afficher ou traiter.
- Dans ton projet : Proprietaire.getAllProprietaires() / ProprietaireController.fetchAllProprietaires()

Extrait de code:

```
// Récupérer tous les propriétaires
public List<Proprietaire> fetchAllProprietaires() {
    return Proprietaire.getAllProprietaires();
}
```

Update (Modifier) :

- Rôle : Modifier les données existantes.
- Dans ton projet : Proprietaire.updateProprietaire() / ProprietaireController.modifyProprietaire()

Extrait de code :

```
// Modifier un propriétaire existant
public boolean modifyProprietaire(int id, String nom, String prenom, String
adresse, String cp, String ville) {
    return Proprietaire.updateProprietaire(id, nom, prenom, adresse, cp,
ville);
}
```

Delete (Supprimer) :

- Rôle : Supprimer des données existantes de la base.
- Dans ton projet : Proprietaire.deleteProprietaire() / ProprietaireController.removeProprietaire()

Extrait de code :

```
// Supprimer un propriétaire
public boolean removeProprietaire(int id) {
    return Proprietaire.deleteProprietaire(id) ;
}
```